CSL253 - Theory of Computation

Tutorial 7

Team Members

1. Harshita Patidar - 12340920
2. Snehal Suhane - 12342100
3. Rithika Sannidhi - 12341900

Question 11

Show that the problem of determining whether a CFG generates some string in 1^* is decidable. Alphabet is $\{0, 1\}$.

Solution:

Any context-free grammar can be transformed into a Pushdown Automaton (PDA) that accepts the same language. To determine whether the CFG generates any string in 1^*, we simulate the corresponding PDA using a three-tape Turing Machine (TM). Here's how each tape functions:

- **Tape 1 (Input Tape):** Contains the input string, like "1111". It is read-only.
- **Tape 2 (Control Tape):** Encodes all transition rules of the PDA. It also tracks the PDA's current state.
- **Tape 3 (Stack Simulation):** Mimics the PDA stack with basic push/pop operations.

1. Setup Phase

- **Tape 1:** Initialized with an input string (e.g., `1111`), with the head at the leftmost symbol.

Tape 1 Initial Configuration

1	1	1	1	···	#	···

- **Tape 2:**
  - *Transition Function Encoding Region:* Before the simulation begins, a fixed encoding of the PDA's transition function (derived from the CFG) is written onto a designated section of this tape. Each transition is encoded in a structured way. Transitions for our PDA:
    * `(q_0, 1, $) → (q_0, $)`
      Continue reading 1s without modifying the stack.
    * `(q_0, 0, $) → (q_r, $)`
      If a 0 is encountered, go to reject state q_r.
    * `(q_0, ϵ, $) → (q_a, $)`
      On finishing input, transition to accepting state q_a.
  - *State Marker Region:* A separate section of Tape 2 records the current state of the PDA (initialized to q_0).

Tape 2 Configuration (initially)

q_0	#	q_0	1	$	q_0	$	#	q_0	0	$	q_r	$	#	q_0	ϵ	$	q_r	$

- **Tape 3:** Begins with just the bottom-of-stack marker $.

Tape 3 Initial Configuration

$	_	_	_

2. Transition Evaluation

Each cycle involves checking the PDA's current configuration and finding a matching rule from Tape 2. The Turing Machine checks:

- Current state from Tape 2
- Current input symbol from Tape 1
- Stack top from Tape 3 (always $)

Once a match is found:

- The PDA's state is updated.
- The head on Tape 1 moves right.
- Stack remains unchanged (no push/pop).

3. Execution Process

- The machine replaces the old state on Tape 2 with the new state.
- Tape 1 head moves one step right.
- Stack stays the same, retaining only $.
- This loop continues until:
  - End of input is reached and current state is accepting, or
  - A `0` is encountered, leading to rejection.

This setup emulates the PDA's behavior for recognizing strings solely of 1s.

4. Conclusion

We have shown that the task of checking whether a CFG generates any string in 1^* is a decidable problem. This was achieved by:

- Designing a PDA that accepts 1^*, and
- Simulating it with a 3-tape Turing Machine that reads the input, uses a minimal fixed stack, and navigates transitions via a lookup table.

Due to the simplicity of the PDA for 1^*, the Turing Machine simulation is direct and deterministic. It halts on all inputs, either accepting if only 1s are read or rejecting otherwise. Thus, we conclude that the intersection of the CFG's language with 1^* is decidable.